

WHAT WILL WE COVER?

Numbers, maths, reading from the keyboard — and how Python handles the types when you combine them.

1

Arithmetic operators

`+`, `-`, `*`, `/`, `//`, `%`, `**` and order of operations

2

`abs()` and `round()`

Two built-ins you will use in almost every calculator

3

`input()`

Read text from the keyboard — and why it is always a string

4

Type conversion

`int()`, `float()`, `str()`, `bool()` — and common type errors

Goal: build a personal calculator that reads two numbers from the keyboard and prints all the arithmetic results.

ARITHMETIC OPERATORS

Python can do maths. Use these seven operators on any numbers or numeric variables.

OperatorNameExampleResult

+ Addition 10 + 3 13

- Subtraction 10 - 3 7

* Multiplication 10 * 3 30

/ Division 10 / 3 3.333... (always float)

// Integer division 10 // 3 3 (whole part only)

% Remainder (modulo) 10 % 3 1 (10 = 3×3 + 1)

** Power 2 ** 8 256

// gives the **whole number** part of a division — useful for splitting things evenly. % gives the **remainder** — useful for checking odd/even.

ARITHMETIC

ARITHMETIC IN ACTION

```
# Shopping total
price    = 2.99
quantity = 4
total    = price * quantity
print(total)      # 11.96

# Age in days
age_years = 25
age_days  = age_years * 365
print(age_days)   # 9125

# Is a number even? remainder is 0 if yes
number = 8
print(number % 2)  # 0 – even!
number = 7
print(number % 2)  # 1 – odd!

# Update a variable with itself
score = 10
score = score + 5   # add 5
print(score)        # 15
```

ORDER OF OPERATIONS

Python follows the same maths rules you learned in school. **Brackets run first, then powers, then multiply and divide, then add and subtract.**

WITHOUT BRACKETS

Default order

```
print(2 + 3 * 4)      # 14
# multiply first: 3*4=12, then 2+12=14

print(10 - 2 ** 3)    # 2
# power first: 2**3=8, then 10-8=2

print(10 / 2 + 3)     # 8.0
# divide first: 10/2=5, then 5+3=8
```



WITH BRACKETS

Brackets change everything

```
print((2 + 3) * 4)    # 20
# brackets first: 2+3=5, then 5*4=20

# Average of three scores – MUST use brackets!
score1 = 80
score2 = 90
score3 = 70

wrong   = score1 + score2 + score3 / 3 # 193.3!
correct = (score1 + score2 + score3) / 3 # 80.0
print(correct)
```



ABS() AND ROUND()

```
# abs() – absolute value, always positive
print(abs(-15))      # 15
print(abs(7))        # 7

temperature = -8
print(abs(temperature)) # 8 ("8 degrees below zero")

# round() – round to nearest whole number
print(round(3.7))     # 4
print(round(3.2))     # 3

# round(number, digits) – keep decimal places
print(round(9.999, 2)) # 10.0
print(round(2.675, 2)) # 2.68

# Practical: price after discount
price = 49.99
discount = 0.15
final = round(price * (1 - discount), 2)
print(final)          # 42.49
```



ARITHMETIC POWERS EVERY APP

Every time you pay online, Python-style arithmetic is running behind the scenes.

E-COMMERCE CHECKOUT

Price × quantity – discount + shipping

```
item_price  = 29.99
quantity    = 3
discount_pct = 10      # 10%
shipping    = 4.90

subtotal = item_price * quantity
discount = subtotal * (discount_pct / 100)
total    = round(subtotal - discount + shipping, 2)
print(f"Total: €{total}")    # Total: €85.87
```



RIDE-SHARING FARE

Base + distance × rate × surge

```
base_fare    = 2.50
distance_km  = 8.3
rate_per_km  = 1.20
surge        = 1.5

price = (base_fare + distance_km * rate_per_km) * surge
price = round(price, 2)
print(f"Fare: €{price}")    # Fare: €18.69
```



MATHS WITH PYTHON

1 You buy 3 coffees at €2.50 each. What is the total? Use `round()`.

2 You have €20. How much change do you get?

3 7 students, 30 biscuits. How many does each get? How many are left over? Use `//` and `%`.

4 What is 2 to the power of 8?

5 Temperature is -12°C. Use `abs()` to print "12 degrees below zero".

Done when: all five values print correctly.

```
# Exercise – Maths with Python
```

```
coffee_price = 2.50
quantity      = 3
```

```
# 1. Total
```

```
# 2. Change from €20
```

```
# 3. Biscuits – each student gets? leftover?
students = 7
biscuits = 30
```

USER INPUT

INPUT()

```
# Ask for the user's name
name = input("What is your name? ")
print("Hello, " + name + "!")

# Ask for their city
city = input("Where are you from? ")
print("Nice! " + city + " is a great city.")

# Use f-strings for cleaner output
age = int(input("How old are you? "))
print(f"Next year you will be {age + 1}.")
```



WHAT HAPPENS WHEN YOU RUN IT

```
What is your name? Sara
Hello, Sara!
Where are you from? Berlin
Nice! Berlin is a great city.
How old are you? 25
Next year you will be 26.
```

USER INPUT

INPUT() ALWAYS RETURNS A STRING

Even if the user types a number, `input()` gives you a **string**. You must convert it before doing any maths.

THE CRASH

TypeError

```
# x This crashes!  
age = input("How old are you? ")  
print(age + 1)  
# TypeError: can only concatenate  
# str (not "int") to str
```



Python cannot add a string and a number. `input()` gave you the string <

INPUT() IN PRACTICE

Here are two small programs that use `input()` — one with text, one with numbers.

TEXT — NO CONVERSION NEEDED

Greeting program

```
name = input("What is your name? ")
city = input("Where are you from? ")

print("Hello, " + name + "!")
print(city + " sounds like a great place.")
```



NUMBERS — MUST CONVERT

Coffee calculator

```
price    = float(input("Price per coffee (€): "))
quantity = int(input("How many coffees? "))

total = round(price * quantity, 2)
print(f"Total: €{total}")
```



The rule is simple: if you are going to do maths with the value, wrap `input()` with `int()` or `float()`. If you only need text, leave it as is.

TYPE CONVERSION

CONVERTING BETWEEN TYPES

```
# str → int
age = int("25")
print(age + 1)           # 26

# str → float
price = float("9.99")
print(price * 2)         # 19.98

# int/float → str (needed before joining with +)
age = 25
print("I am " + str(age) + " years old")

# int → float and back
x = float(5)             # 5.0
y = int(3.9)             # 3 ← cut off, NOT rounded!

# Anything → bool
print(bool(1))           # True
print(bool(0))           # False
print(bool("hello"))     # True
print(bool(""))          # False ← empty string is False
```

TYPE CONVERSION

COMMON TYPE ERRORS AND FIXES

TYPEERROR — MIXING STR AND INT

Cannot add string and number

```
# ✗ Crashes
name = "Sara"
age = 25
print(name + age)

# ✓ Fix — convert age to string
print(name + " is " + str(age))    # Sara is 25
# ✓ Or use a comma (no conversion needed)
print(name, "is", age)            # Sara is 25
```



VALUEERROR — INVALID CONVERSION

Cannot convert letters to a number

```
# ✗ Crashes
score = "90"
print(score + 10)    # TypeError!
```

THE WRONG TYPE CAUSES CRASHES

Python cannot guess what you mean. If you try to add a string and a number it stops immediately. The fix is always the same — convert first.

THE PROBLEM

input() gives you text

```
age = input("How old are you? ")
# age is the string "25", not the number 25

print(age + 1)      # TypeError – crashes!
print(age * 2)      # prints "2525" – wrong!
```



<p>When you multiply a string by a number, Python repeats the text. That is not what y

USER INPUT AND CONVERSION

1 Ask for the user's **name** with `input()`. Print "Hello, [name]!"

2 Ask for their **age** (as int). Print how old they will be in 10 years.

3 Ask for a **price** (as float). Print the price with **21% VAT** added, rounded to 2 decimal places.

4 Ask for two numbers. Print their **sum, difference, product, and quotient**.

Done when: your program runs, accepts input, and prints all results without crashing.

```
# Exercise – User Input & Conversion
```

```
# 1. Name greeting
```

```
name =
```

```
print()
```

```
# 2. Age in 10 years
```

```
age =
```

```
print()
```

```
# 3. Price with VAT
```

```
price =
```

```
vat_rate = 0.21
```

```
print()
```

```
# 4. Two numbers
```

```
a =
```

Watch out for these

1

Using `input()` result as a number without converting

`age = input("Age: ")` → `age + 1` crashes. Always wrap with `int()` or `float()`.

2

Forgetting brackets in averages

`score1 + score2 + score3 / 3` only divides the last number. Write `(score1 + score2 + score3) / 3`.

3

Expecting `int()` to round

`int(3.9)` gives `3`, not `4`. Use `round()` when you want rounding, `int()` only cuts the decimal.

4

Capitalised built-ins

`Round()`, `Abs()`, `Int()` all crash with `NameError`. Python built-ins are always lowercase.

For next class

- 1 **All operators** — create two variables `a = 17`, `b = 5`. Print the result of all seven operators (+, -, *, /, //, %, **) with a label for each.
- 2 **Average calculator** — create three test scores and calculate the average using brackets. Round to 1 decimal place.
- 3 **abs() practice** — create two temperature variables (e.g. yesterday = -3, today = 7). Print the difference using abs() so it is always positive.
- 4 **Discount calculator** — ask the user for a price and a discount percentage. Print the original price, the discount amount, and the final price. Round everything to 2 decimal places.
- 5 **BMI calculator** — ask the user for weight (kg) and height (m). Calculate BMI as `weight / height ** 2`. Print the result rounded to 1 decimal place.
- 6 **Seconds converter** — ask for a number of seconds. Print how many full hours, minutes, and leftover seconds that is. Use `//` and `%`.
- 7 **Personal calculator** — ask the user for two numbers. Print their sum, difference, product, quotient (rounded to 2 dp), integer division, remainder, and the first number to the power of the second.

YOU CAN DO MATHS WITH PYTHON

- 1 `+` `-` `*` `/` `//` `%` `**`
- 2 Order of operations
- 3 `abs()` · `round()`
- 4 `input()` always returns str
- 5 `int()` · `float()` · `str()` · `bool()`

Next lesson: strings, string methods, and f-strings.